Rewarding Learning

General Certificate of Secondary Education
2026

Digital Technology

Unit 4

Digital Development Concepts

[GDG41]

TUESDAY 2 JUNE, MORNING

MARK SCHEME

General Marking Instructions

Introduction

Mark schemes are published to assist teachers and students in their preparation for examinations. Through the mark schemes teachers and students will be able to see what examiners are looking for in response to questions and exactly where the marks have been awarded. The publishing of the mark schemes may help to show that examiners are not concerned about finding out what a student does not know but rather with rewarding students for what they do know.

The Purpose of Mark Schemes

Examination papers are set and revised by teams of examiners and revisers appointed by the Council. The teams of examiners and revisers include experienced teachers who are familiar with the level and standards expected of students in schools and colleges.

The job of the examiners is to set the questions and the mark schemes; and the job of the revisers is to review the questions and mark schemes commenting on a large range of issues about which they must be satisfied before the question papers and mark schemes are finalised.

The questions and the mark schemes are developed in association with each other so that the issues of differentiation and positive achievement can be addressed right from the start. Mark schemes, therefore, are regarded as part of an integral process which begins with the setting of questions and ends with the marking of the examination.

The main purpose of the mark scheme is to provide a uniform basis for the marking process so that all the markers are following exactly the same instructions and making the same judgements in so far as this is possible. Before marking begins a standardising meeting is held where all the markers are briefed using the mark scheme and samples of the students' work in the form of scripts. Consideration is also given at this stage to any comments on the operational papers received from teachers and their organisations. During this meeting, and up to and including the end of the marking, there is provision for amendments to be made to the mark scheme. What is published represents this final form of the mark scheme.

It is important to recognise that in some cases there may well be other correct responses which are equally acceptable to those published: the mark scheme can only cover those responses which emerged in the examination. There may also be instances where certain judgements may have to be left to the experience of the examiner, for example, where there is no absolute correct response – all teachers will be familiar with making such judgements.

Listing

Where **one** response is required to gain a mark, candidates will not gain credit if a correct response is given alongside one or more incorrect responses. This is referred to as listing.

		AVAILABLE MARKS
1	**(a)** **B** American Standard Code for Information Interchange. [1]	
	(b) **A** There are a maximum of 128 characters. [1]	
	(c) **A** Unicode can represent more characters than 7-bit ASCII but less characters than 8-bit ASCII. [1]	
	(d) **C** The file is 1000 Megabytes in size. [1]	

(e)

Bit Pattern	Unit of Data
0	**Bit**
10101010	Byte [1]
1010	Nibble [1]

[2] 6

2 **(a)** **C** 12 13 14 15 16 17 [1]

(b) **B** 6 4 2 [1]

(c) **A** # # # # # [1] 3

3 **(a)**

Statement	TRUE/FALSE
User-defined functions make programs less efficient.	FALSE/F [1]
User-defined functions are re-useable within a program.	TRUE/T [1]
User-defined functions make testing a program easier.	TRUE/T [1]
User-defined functions cannot return values to the main program.	FALSE/F [1]

[4]

(b) **(i)** Strings can be joined together using a **CONCATENATE** [1] function. To extract part of a string a **SUBSTRING** [1] function can be used. A **LENGTH** [1] function can be used to determine the number of **CHARACTERS** [1] in a string. [4]

(ii)
```
average = (t1 + t2 + t3 + t4 + t5)/5.0
OUTPUT average
IF (average [1]      >[1]      25.0 [1])
    Message = "CODE 1: Too hot"
ELSE
    Message = [1]  "CODE 2: No Action Required" [1]
ENDIF
OUTPUT Message [1]
```
Must use correct variable named 'average'
Allow words 'is greater than' [1]
Must have 'Message=' for [1] capital not necessary [6]

(c) **(i)** Any **two** from:
- Breaks the rules of the language [1]
- Code does not conform to the (grammar) rules of the language [1]
- Program cannot run/compile/syntax errors must be corrected before the program can run [1] Do not accept 'crash'

AVAILABLE MARKS

- Suitable example, e.g. leave out a semi-colon or misspelling/ punctuation error [1]
- Occurs at compile time/before execution [1] [2]

(ii)

How the feature can help a programmer
Auto indentation Any **two** from: • Consistent formatting of code blocks/enforces a shared style [1] • Improves code readability [1] • Helps identify scope of variables(what variable is available where) [1] • Can assist debugging/error detection [1] • Indicates the structure/hierarchy of the code [1] • Shows relationships between different blocks of code e.g. loops, if-statements [1]
Code completion Any **two** from: • Accelerates the coding process [1] • Reduces typographical errors/human error/chance of errors [1] • Helps programmers adhere to coding standards [1] • Promotes the use of consistent naming conventions [1] • Minimises the need to recall specific syntax/helps programmer with syntax [1] • Provides Tooltips (signature help/parameter help/quick info)/Provides popup/lists of available functions/statements/constants/values/when entering statements, the correct syntax and arguments are shown/it is predictive/predicts the next word/argument/variable/parameter [1]

[4] 20

4 **(a)** Conversion work – accept divide by two or place value [1]
00110010 [1]/allocate [1] for 7 bits 0110010/6 bits 110010 [2]

(b) **(i)** For any correct conversion : 0010 = 2/1111=F [1]
For correct answer 2F [1] [2]

(ii) Conversion work [1] 47 [1] [2]

(c)

		1	0	0	1	0	1	0	0
+		1	1	0	0	0	1	1	1
Carry		-	-	-	-	**1**	-	-	-
Answer	(1)	0	1	0	1	1	**0**	1	1

[1] for each entry, [1] for circling overflow [4] 10

5 **(a)**

Description	**Boolean operator**
The output is true only if all inputs are true.	**AND** [1]
The output is true if one or all of the inputs are true.	**OR** [1]
The output is true if the input is false and the output is false if the input is true.	**NOT** [1]

[3]

(b)

A	B	C= NOT [1] (A AND B) Brackets required [1]
0	0	1 [1]
0	1	1 [1]
1	0	1 [1]
1	1	0

[5]

AVAILABLE MARKS

8

6 Level 0 [0]
Answer is not worthy of credit.

Level 1 ([1]–[2])
The candidate correctly refers to one [1] or two [2] of the following: how an array is created or declared; how individual elements in the array can be accessed and changed.
The candidate makes limited use of spelling, punctuation and grammar. The meaning of the text is not always clear. The candidate displays a limited form and style appropriate to the question. The organisation of the answer is limited.

Level 2 ([3]–[4])
The candidate correctly describes one [3] or two [4] of the following: how an array is created or declared; how individual elements in the array can be accessed and changed.
The candidate makes satisfactory use of spelling, punctuation and grammar. The meaning of the text is usually clear. The candidate demonstrates a satisfactory form and style appropriate to the question. The organisation of the answer is satisfactory.

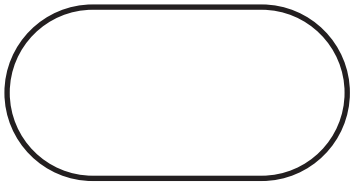
Level 3 ([5]–[6])
The candidate fully describes each of the following: how an array is created or declared; how individual elements in the array can be accessed and changed. Examples from marks are used to support the answer.
The candidate uses a good standard of spelling, punctuation and grammar. The meaning of the text is always clear. The candidate demonstrates a good standard of form and style appropriate to the question. The organisation of the answer is good.
Two good descriptions with one supporting example [5]
Two good descriptions with two supporting examples [6]
Answers may include some **(not all)** of the following:

- An array is a data structure, it stores multiple values of the same data type.
- Array declaration – An array is created by specifying a data type and a size (e.g. int[] marks = new int[10];).
- Array initialisation – Arrays can also be created with preset values (e.g. int[] marks = {60, 59, 50, 55, 56, 58, 60, 60, 52, 50};).
- Index-based access – Individual elements are accessed using an index starting at 0 (e.g. marks[0]). Zero index.
- Accessing values – Accessing an element returns the value stored at that index (e.g. x = marks[2];).
- Changing values – Elements are updated by assigning a new value to a specific index (e.g. marks[3] = 10;).
- Fixed size behaviour – Once created, the array's size cannot change; only the contents at each index can be modified.
- Processing the elements in an array – an example of processing data using a loop.
- Reference to improved efficiency of programming or more efficient storage and processing.

[6]

6

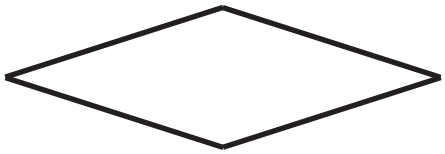
7 (a)



1. START/STOP/END/ TERMINAL/INITIALISE/ TERMINATOR [1]



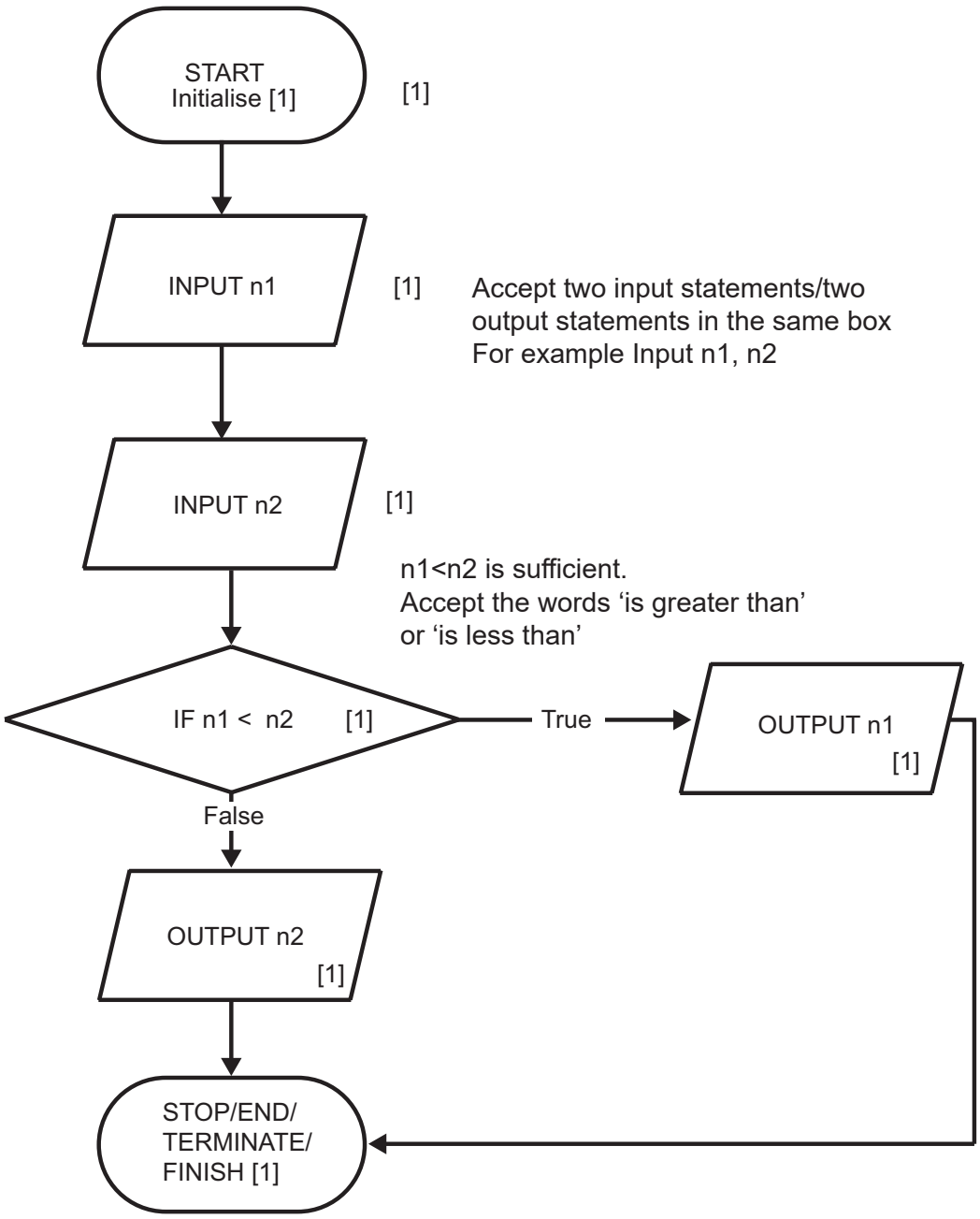
2. INPUT/OUTPUT [1]
Accept other or both



3. DECISION(BOX) [1] [3]

AVAILABLE MARKS

(b)



[The condition above can be replaced with n1<n2/n2<n1/n2>n1.
Associated output statements must be correct in relation to the condition used.] [7]

(c)

Name of symbol	Drawing of symbol
PROCESS/OPERATION [1]	[1]

[2]

AVAILABLE MARKS

12

AVAILABLE MARKS

8 **(a)** Filtering/removing (unnecessary) details/data/information (about the problem) [1] not required for the solution [1] [2]

(b)

Statement	TRUE/FALSE
Pseudo-code is a graphical representation of a solution.	FALSE/F [1]
Pseudo-code uses keywords and control structures in a similar way to a programming language.	TRUE/T [1]
Pseudo-code is language independent.	TRUE/T [1]
Pseudo-code is the source code for a solution.	FALSE/F [1]

[4]

(c) **(i)**

Criteria	Validation check
The password must begin with 3 letters followed by 3 numbers	FORMAT [1]
The password must be at least 8 characters long	LENGTH [1]

[2]

(ii)

```
OUTPUT "Enter username"
INPUT username
OUTPUT "Enter password"
INPUT password
IF username="HENRYK156" [1] AND [1] password="ADM967r!" [1]
      OUTPUT "Log in successful" (or suitable message)[1]
ELSE
      OUTPUT "Login details not correct" (or suitable message) [1]
```

Note that alternative solutions using correct conditions and nested if statements are acceptable if correct output is achieved.

1. Username check [1]
2. AND/non exclusive IF [1]
2. Password check [1]
4. *Successful* output statement in the correct position [1]
5. *Unsuccessful* output statement in the correct position [1] [5]

(d) **(i)** To prevent information from being lost when the program terminates/ Information is lost when the program closes/is not running [1]
Any **one** from:
File provides permanent storage [1]
Saving to a file means the data will remain/is persistent/is not lost/can be used when the program runs again [1] [2]

(ii) Write data to a file [1]
Append data to a file [1]
Read data from file [1] [2]

17

9 **(a)**

Variable	Data type
DogSize	Character/Char [1]
GroomCost	Double/Float/Real [1]

[2]

AVAILABLE MARKS

(b)

```
OUTPUT "Enter the size of the dog S, M, L"
INPUT DogSize
IF (DogSize='L')
        GroomCost=80.0              [1] For correct condition with matching cost assignment
    ELSE IF (DogSize='M')
            GroomCost=65.0          [1] For correct condition with matching cost assignment
        ELSE IF (DogSize='S')       (Also accept ELSE GroomCost=45.0 or equivalent)
            GroomCost=45.0          [1] For correct condition with matching cost assignment
        END IF
    END IF
END IF
```

Separate IF-statements are acceptable provided the condition and matching cost assignment are correct. Assignment statements must be structured correctly.

Alternative credit for "NO"
IF DogCollected = "NO" [1]
OUTPUT GroomCost/TotalGroomCost[1] [3]

(c)

```
TotalGroomCost=GroomCost
OUTPUT "Is dog collection needed? "
INPUT DogCollected
IF DogCollected = "YES" [1]
    OUTPUT  "Enter the distance to be travelled" (or suitable message) [1]
    INPUT  DogDistance [1]
        IF DogDistance [1]    <= [1]   5 [1]
            TotalGroomCost = TotalGroomCost/GroomCost [1] +  6.00
        ELSE
            OUTPUT  "Collection not possible" (or suitable message) [1]
        ENDIF
ENDIF
OUTPUT TotalGroomCost                                              [8]
```

13

Must be "YES" capitals not necessary.

Must use variable names supplied.

AVAILABLE MARKS

10 (a)

5	20	40	10	30

[1]

5	20	40	10	30

[1]

5	10	20	40	30

[1]

5	10	20	30	40

[1] [4]

(b)

```
n=5
FOR X = 0 TO  4 [1] / n -1 [1]
    FOR Y = 0 TO  n -X -1[2]   4 [1] n -1 [1]
        IF numbers[Y]  > numbers[Y+1] [1]
            Temp = numbers[Y]
            numbers[Y] = numbers[Y+1]
            numbers[Y+1] = Temp [1]
        END IF
    END FOR
END FOR
```

[5]

Do not accept 5 for loop upper limit

9

11 (a) Any **two** from:

- Checks underlying structure/code/logic [1]
- Tester needs to have a detailed knowledge of the development/code/ carried out by the programmer [1]
- Specific inputs are selected to ensure the appropriate output is produced [1]
- Tested using a dry run [1]
- Trace tables used to examine any variable [1]
- Finds logic errors [1]

[2]

(b)

Test Number	Reason for test	Test data	Expected outcome
1.	Valid value Correct data [1]	18	Value Accepted
2.	Null value [1]	Press Enter Key	Value Rejected
3.	Extreme Value	17 or 20 [1]	Value Accepted
4.	Invalid value [1]	35	Value Rejected [1]

[5]

(c) (i) Line number 5 [1] is incorrect. [1]

(ii) IF (value>=17 AND [1] value <=20) [1] [2]

10

		AVAILABLE MARKS
12 **(a)**	Any **two** from: • client/customer [1] • (end) user [1] • developer/author of code/programmer/development team [1] [2]	
(b) **(i)**	Any **two** from: • Gives an indication of reliability/robustness/strengths and weaknesses [1] • Shows how well the system works/performs/shows what functions and what doesn't [1] • Changes or improvements made [1] [2]	
(ii)	Any **two** from: • Used as a checklist [1] • Compare completed system to user requirements [1] • To ensure requirements met/to ensure the system is complete/ to ensure no parts of system are missing [1] [2]	6
	Total	**120**